devp2p 프로토콜 스택 분석

개념적 배경

devp2p가 필요한 이유

devp2p는 Ethereum의 네트워크 계층 프로토콜로, 노드 간의 안전하고 효율적인 통신을 위해 설계되었습니다. 단순한 TCP 연결과 달리, devp2p는 다음과 같은 고급 기능들을 제공합니다:

핵심 요구사항:

- **암호화 통신**: 모든 메시지가 암호화되어 전송
- **인증**: 피어의 신원 확인 및 검증
- **프로토콜 멀티플렉싱**: 하나의 연결에서 여러 프로토콜 동시 실행
- **압축**: 네트워크 대역폭 효율성
- **연결 관리**: 자동 재연결 및 에러 처리

다른 접근법과의 비교

기존 방식의 한계:

- **평문 통신**: 보안 취약점 존재
- **단일 프로토콜**: 확장성 부족
- **중앙화된 인증**: 단일 실패 지점

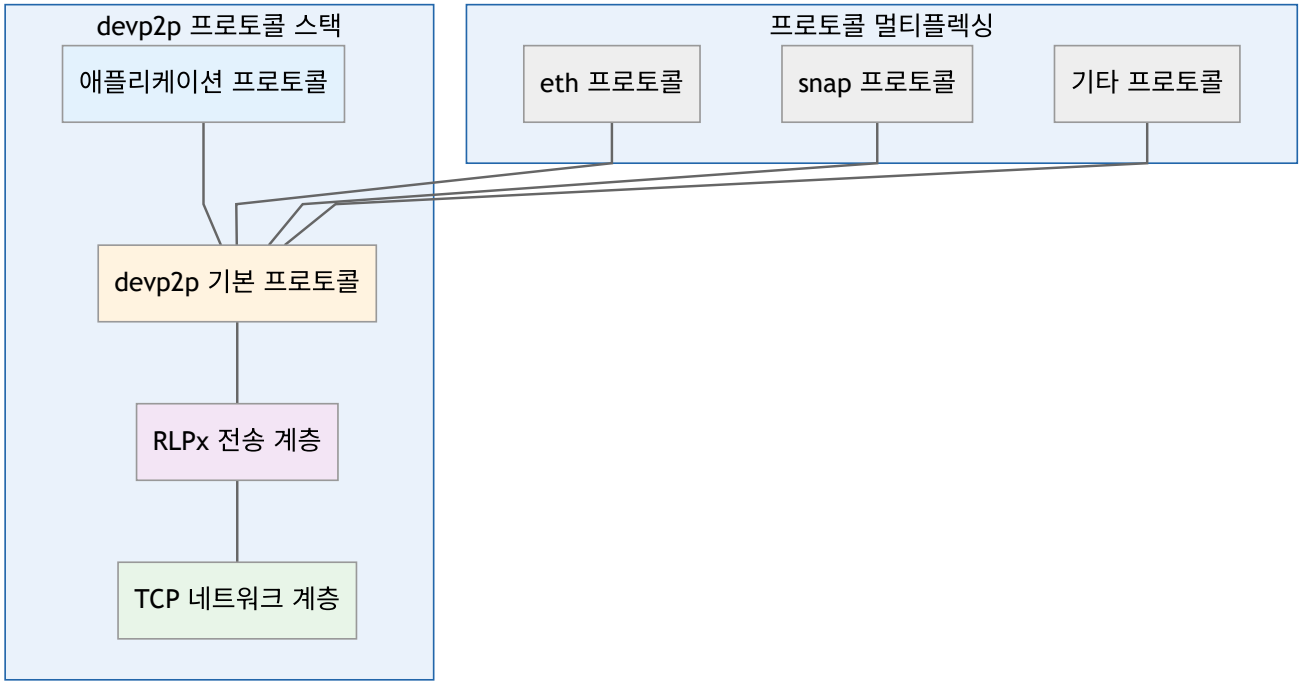
devp2p의 혁신:

- **RLPx 암호화**: ECIES 기반 종단간 암호화
- **프로토콜 협상**: 동적 기능 협상

- **분산 인증**: 공개키 기반 신원 확인

구현 분석

핵심 아키텍처



1. 서버 구조체 (p2p/server.go)

```
type Server struct {
Config // 서버 설정
// 연결 관리
listener net.Listener
ourHandshake *protoHandshake // 우리의 핸드셰이크 정보
// 디스커버리 시스템
```

```
nodedb *enode.DB

localnode *enode.LocalNode

discv4 *discover.UDPv4

discv5 *discover.UDPv5

discmix *enode.FairMix

dialsched *dialScheduler

// 이벤트 채널들

quit  chan  struct{}

addtrusted  chan *enode.Node

removetrusted  chan *enode.Node

peerOp  chan peerOpFunc

delpeer  chan peerDrop

checkpointPostHandshake  chan *conn

checkpointAddPeer  chan *conn

}
```

2. 연결 구조체

```
type conn struct {

fd net.Conn // 실제 네트워크 연결

transport // RLPx 전송 인터페이스

node *enode.Node // 연결된 노드 정보

flags connFlag // 연결 타입 플래그
```

```
cont  chan  error  // 에러 통신 채널

caps []Cap // 지원하는 기능들

name  string  // 피어 이름

}


// 연결 타입 플래그

const (

dynDialedConn connFlag = 1 << iota // 동적 다이얼 연결

staticDialedConn  // 정적 다이얼 연결

inboundConn  // 인바운드 연결

trustedConn  // 신뢰할 수 있는 연결

)
```

3. 전송 인터페이스

```
type transport interface {

// 두 단계 핸드셰이크

doEncHandshake(prv *ecdsa.PrivateKey) (*ecdsa.PublicKey, error)

doProtoHandshake(our *protoHandshake) (*protoHandshake, error)

// 메시지 읽기/쓰기 (암호화 핸드셰이크 완료 후)

MsgReadWriter

// 연결 종료

close(err  error)
```

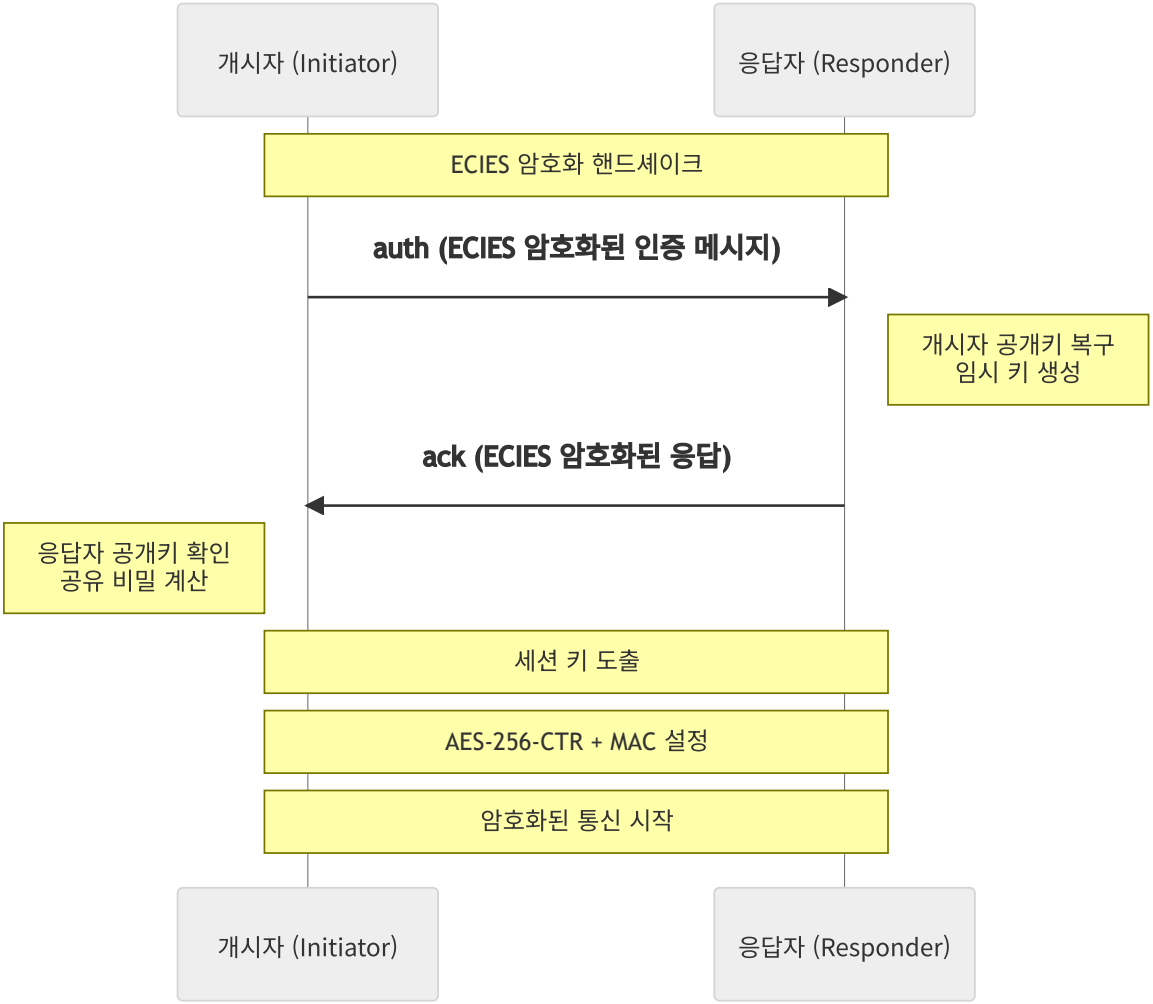
```
}
```

RLPx 암호화 핸드셰이크

1. RLPx 연결 구조 (p2p/rlpx/rlpx.go)

```
type Conn struct {
dialDest *ecdsa.PublicKey // 다이얼 대상 (클라이언트만)
conn net.Conn // 기본 네트워크 연결
session *sessionState // 세션 상태
// Snappy 압축 버퍼
snappyReadBuffer []byte
snappyWriteBuffer []byte
}

type sessionState struct {
enc cipher.Stream // 송신 암호화 스트림
dec cipher.Stream // 수신 복호화 스트림
egressMAC hashMAC // 송신 MAC
ingressMAC hashMAC // 수신 MAC
rbuf readBuffer
wbuf writeBuffer
}
```

2. 암호화 핸드셰이크 과정



3. 핸드셰이크 구현 세부사항

인증 메시지 구조:

```
type authMsgV4 struct {
gotPlain  bool  // rlp 디코딩 후 설정
Signature [sigLen]byte
InitiatorPubkey [pubLen]byte
Nonce [shaLen]byte
```

```
Version  uint

// 무시되는 추가 필드들 (호환성)

Rest []rlp.RawValue `rlp:"tail"`

}
```

응답 메시지 구조:

```
type authRespV4 struct {

RandomPubkey [pubLen]byte

Nonce [shaLen]byte

Version  uint

// 무시되는 추가 필드들 (호환성)

Rest []rlp.RawValue `rlp:"tail"`

}
```

4. MAC 계산 메커니즘

```
type hashMAC struct {

cipher cipher.Block // AES 블록 암호

hash hash.Hash // Keccak-256 해시

aesBuffer [16]byte  // AES 버퍼

hashBuffer [32]byte  // 해시 버퍼

seedBuffer [32]byte  // 시드 버퍼

}
```

```go
func (h *hashMAC) computeHeader(header []byte) []byte {
// 헤더 MAC 계산 로직
// 1. 이전 MAC 상태 업데이트
// 2. 헤더 데이터 해시
// 3. AES 암호화 적용
// 4. 최종 MAC 값 반환
}
```

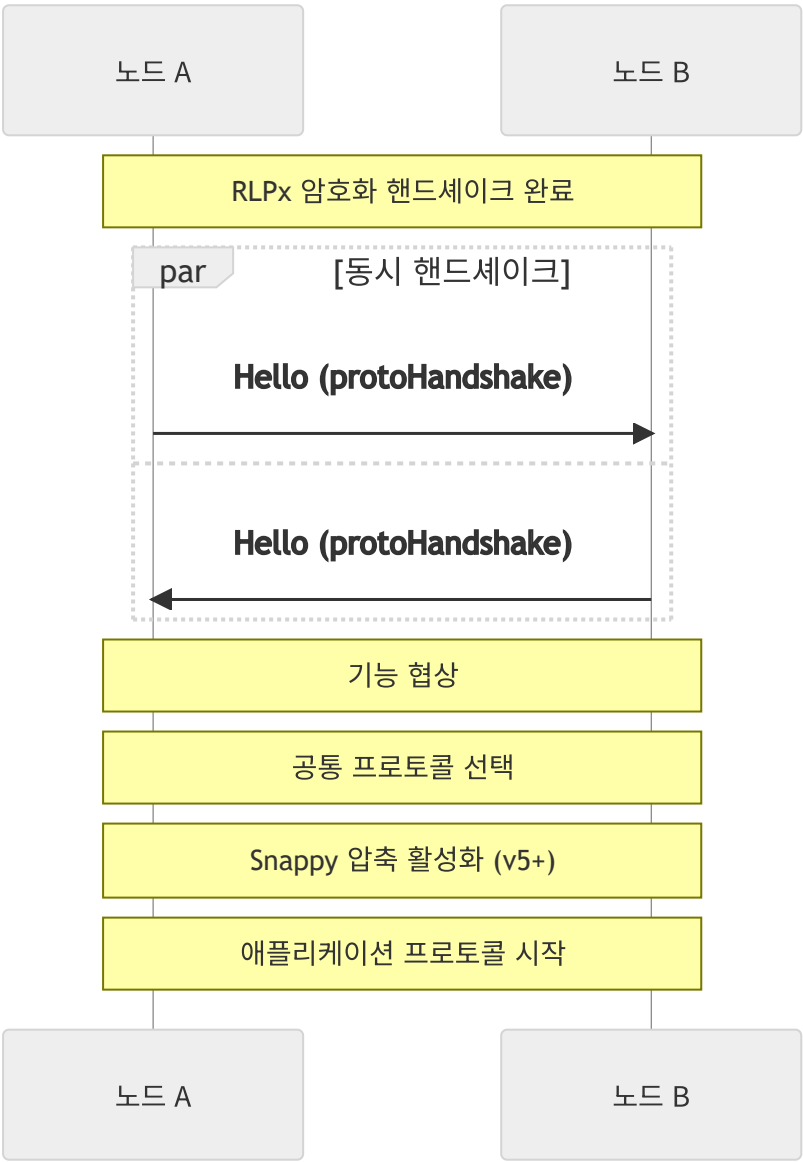
프로토콜 핸드셰이크

1. 핸드셰이크 구조

```go
type protoHandshake struct {
Version  uint64  // 프로토콜 버전
Name  string  // 클라이언트 이름
Caps []Cap // 지원하는 기능들
ListenPort  uint64  // 리스닝 포트
ID []byte  // secp256k1 공개키
// 호환성을 위한 추가 필드들
Rest []rlp.RawValue `rlp:"tail"`
}
```

```go
type Cap struct {
Name  string  // 프로토콜 이름 (예: "eth")
Version  uint  // 프로토콜 버전 (예: 68)
}
```

2. 핸드셰이크 과정



3. 기능 협상 로직

```go
func (srv *Server) setupConn(c *conn, flags connFlag, dialDest *enode.Node) erro
```

```
// RLPx 암호화 핸드셰이크

remotePubkey, err := c.doEncHandshake(srv.PrivateKey)

if err != nil {

return &encHandshakeError{err: err}

}

// 프로토콜 핸드셰이크

phs, err := c.doProtoHandshake(srv.ourHandshake)

if err != nil {

return &protoHandshakeError{err: err}

}

// 신원 검증

if id := c.node.ID(); !bytes.Equal(crypto.Keccak256(phs.ID), id[:]) {

return DiscUnexpectedIdentity

}

// 기능 매칭

c.caps, c.name = phs.Caps, phs.Name

return nil

}
```

프로토콜 멀티플렉싱

1. 메시지 라우팅

```
type Peer struct {
rw *conn // 기본 연결
running map[string]*protoRW // 실행 중인 프로토콜들
// 제어 채널들
protoErr chan error
closed chan struct{}
pingRecv chan struct{}
disc chan DiscReason
}

type protoRW struct {
Protocol
offset uint64 // 메시지 코드 오프셋
in chan Msg // 수신 메시지 채널
closed <-chan struct{}
wstart <-chan struct{}
werr chan<- error
}
```

2. 메시지 디스패치

```
graph TD
A[수신 메시지] --> B{메시지 코드 확인}
```

```
B -->|0x00-0x0F|  C[devp2p 기본 프로토콜]

B -->|0x10+|  D[프로토콜 오프셋 계산]

D -->  E{프로토콜 식별}

E -->|eth|  F[eth 프로토콜 핸들러]

E -->|snap|  G[snap 프로토콜 핸들러]

E -->|기타|  H[기타 프로토콜 핸들러]

C -->  I[핸드셰이크/핑퐁/연결해제]

F -->  J[블록/트랜잭션 처리]

G -->  K[스냅샷 동기화]

H -->  L[커스텀 처리]

style A fill:#e1f5fe

style C fill:#fff3e0

style F fill:#e8f5e8

style G fill:#f3e5f5
```

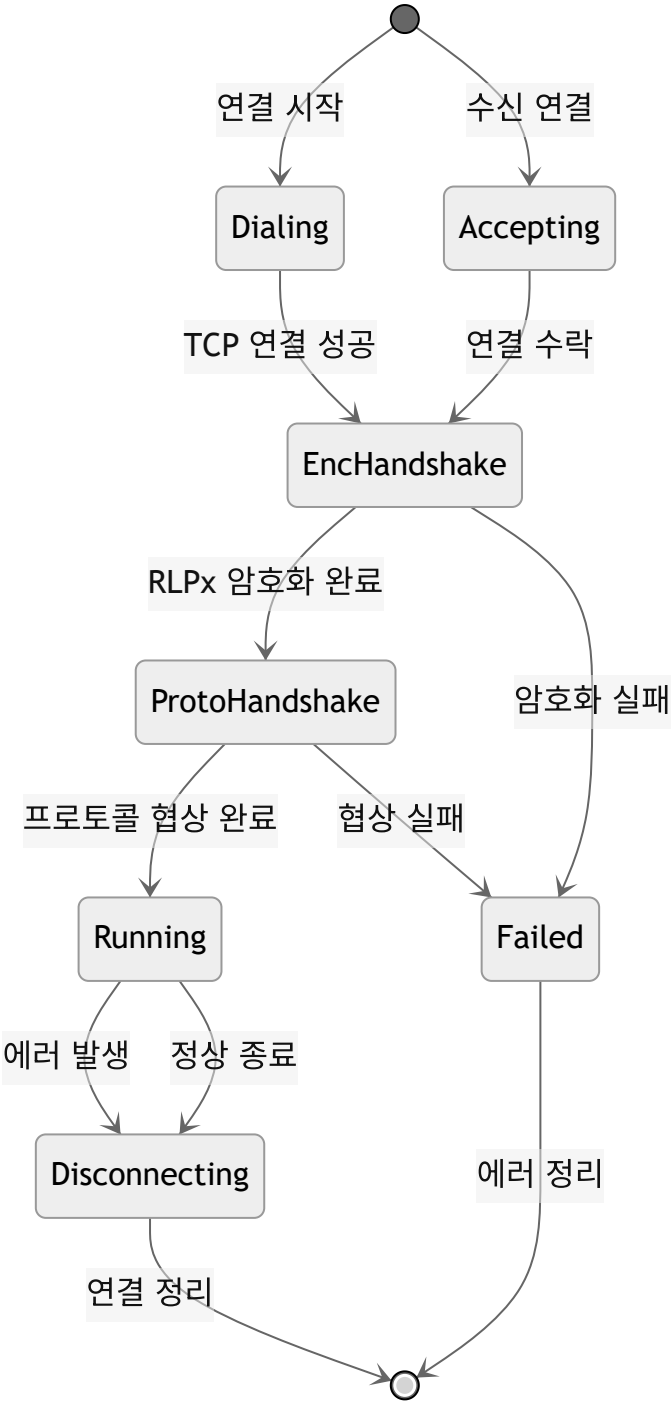
3. 프로토콜 등록 및 시작

```go
func (p *Peer) startProtocols(writeStart <-chan  struct{}, writeErr  chan<- erro

p.wg.Add(len(p.running))

for  _, proto := range  p.running {

proto := proto

proto.closed = p.closed

proto.wstart = writeStart
```

```
proto.werr = writeErr

go  func() {

defer  p.wg.Done()

err := proto.Run(p, proto)

if  err == nil {

p.log.Trace("Protocol returned", "protocol", proto.Name)

} else {

p.log.Trace("Protocol failed", "protocol", proto.Name, "err", err)

p.protoErr <- err

}

}()

}

}
```

연결 생명주기 관리

1. 연결 설정 과정



2. 에러 처리 메커니즘

```
type DiscReason uint


const (
DiscRequested DiscReason = iota
DiscNetworkError
```

```
DiscProtocolError

DiscUselessPeer

DiscTooManyPeers

DiscAlreadyConnected

DiscIncompatibleVersion

DiscInvalidIdentity

DiscQuitting

DiscUnexpectedIdentity

DiscSelf

DiscReadTimeout

DiscSubprotocolError = 0x10

)


func (d DiscReason) String() string {

switch  d {

case  DiscRequested:

return  "disconnect requested"

case  DiscNetworkError:

return  "network error"

// ... 기타 에러 타입들

}

}
```

3. 자동 재연결 시스템

```
type dialScheduler struct {
setupFunc dialSetupFunc
wg sync.WaitGroup
cancel context.CancelFunc
ctx context.Context
nodesIn  chan *enode.Node
doneCh  chan *dialTask
addStaticCh  chan *enode.Node
remStaticCh  chan *enode.Node
addPeerCh  chan *conn
remPeerCh  chan *conn
}

func (d *dialScheduler) loop(it enode.Iterator) {
var (
dialing = make(map[enode.ID]*dialTask)
peers = make(map[enode.ID]*conn)
static = make(map[enode.ID]*dialTask)
)
// 다이얼 스케줄링 루프
for {
```

```
select {
case node := <-d.nodesIn:
// 새 노드 다이얼 시도
case task := <-d.doneCh:
// 다이얼 작업 완료 처리
case c := <-d.addPeerCh:
// 새 피어 추가
case c := <-d.remPeerCh:
// 피어 제거
}
}
}
```

성능 최적화

1. Snappy 압축

```
func (c *Conn) SetSnappy(snappy bool) {
if snappy {
c.snappyReadBuffer = []byte{}
c.snappyWriteBuffer = []byte{}
} else {
c.snappyReadBuffer = nil
```

```
c.snappyWriteBuffer = nil

}

}


func (c *Conn) Write(code  uint64, data []byte) (uint32, error) {

if  c.snappyWriteBuffer != nil {

// Snappy 압축 적용

c.snappyWriteBuffer = snappy.Encode(c.snappyWriteBuffer[:0], data)

data = c.snappyWriteBuffer

}

return  c.writeFrame(code, data)

}
```

압축 효과:

- 일반적으로 20-40% 크기 감소
- CPU 오버헤드는 최소화
- 네트워크 대역폭 절약

2. 버퍼 관리

```
type readBuffer struct {

data []byte

pos  int

}
```

```go
func (r *readBuffer) read(conn io.Reader, n int) ([]byte, error) {
// 버퍼 크기 확인 및 확장
if cap(r.data) < n {
r.data = make([]byte, n)
} else {
r.data = r.data[:n]
}
// 데이터 읽기
_, err := io.ReadFull(conn, r.data)
return r.data, err
}
```

3. 연결 풀링

```go
const (
defaultMaxPendingPeers = 50 // 최대 대기 중인 피어 수
defaultDialRatio = 3 // 다이얼 비율
inboundThrottleTime = 30 * time.Second // 인바운드 제한 시간
)

// IP별 연결 제한
func (srv *Server) checkInboundConn(remoteIP netip.Addr) error {
```

```
if netutil.IsLAN(remoteIP) {
return nil
}
now := srv.clock.Now()
srv.inboundHistory.expire(now, inboundThrottleTime)
if srv.inboundHistory.contains(remoteIP.String()) {
return errors.New("too many attempts")
}
srv.inboundHistory.add(remoteIP.String(), now)
return nil
}
```

실제 사용 예시

1. 서버 설정 및 시작

```
// 서버 설정
config := p2p.Config{
MaxPeers: 50,
MaxPendingPeers: 10,
DialRatio: 3,
NoDiscovery: false,
DiscoveryV5: true,
```

```
Name: "geth/v1.10.0",

BootstrapNodes: []*enode.Node{

// 부트노드들...

},

StaticNodes: []*enode.Node{

// 정적 노드들...

},

Protocols: []p2p.Protocol{

// 지원할 프로토콜들...

},

}


// 서버 생성 및 시작

server := &p2p.Server{Config: config}

if err := server.Start(); err != nil {

log.Fatal("Failed to start P2P server:", err)

}

defer server.Stop()
```

2. 커스텀 프로토콜 구현

```
func NewMyProtocol() p2p.Protocol {

return p2p.Protocol{
```

```
Name: "myproto",
Version: 1,
Length: 10, // 메시지 코드 범위
Run: runMyProtocol,
}
}

func runMyProtocol(peer *p2p.Peer, rw p2p.MsgReadWriter) error {
for {
msg, err := rw.ReadMsg()
if err != nil {
return err
}
switch msg.Code {
case 0x00:
// 메시지 타입 0 처리
handleMessage0(msg)
case 0x01:
// 메시지 타입 1 처리
handleMessage1(msg)
default:
return fmt.Errorf("unknown message code: %d", msg.Code)
}
```

```
}

}
```

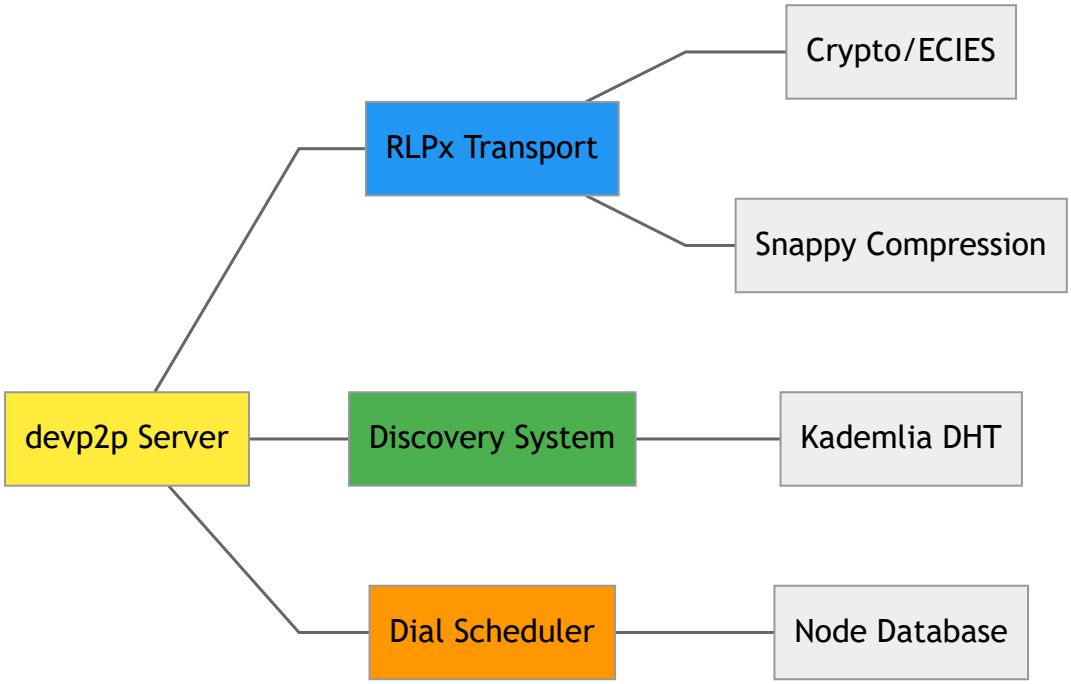
3. 메시지 송수신

```
// 메시지 전송
func sendMessage(rw p2p.MsgReadWriter, code uint64, data interface{}) error {
return p2p.Send(rw, code, data)
}

// 메시지 수신
func receiveMessage(rw p2p.MsgReadWriter) error {
msg, err := rw.ReadMsg()
if err != nil {
return err
}
defer msg.Discard()
var data MyMessageType
if err := msg.Decode(&data); err != nil {
return err
}
// 메시지 처리
return processMessage(data)
```

```
}
```

관련 시스템

의존성 관계



상호작용 패턴

1. **Discovery → devp2p**: 발견된 노드들을 연결 대상으로 제공
2. **devp2p → Application**: 설정된 연결을 애플리케이션 프로토콜에 제공
3. **RLPx → devp2p**: 암호화된 전송 계층 제공
4. **Scheduler → devp2p**: 연결 관리 및 재시도 로직

영향 범위

직접 영향:

- 모든 P2P 통신의 기반

- 네트워크 보안 수준 결정
- 연결 안정성 및 성능

간접 영향:

- 블록체인 동기화 속도
- 트랜잭션 전파 효율성
- 전체 네트워크 안정성

문제 해결 가이드

일반적인 문제들

1. **핸드셰이크 실패**

```
증상: "rlpx enc error" 또는 "rlpx proto error"

원인:

- 네트워크 연결 불안정

- 방화벽 차단

- 프로토콜 버전 불일치

해결방법:

- 네트워크 연결 확인

- 포트 개방 상태 점검

- 클라이언트 버전 업데이트
```

2. **연결 수 제한**

```
증상: "too many peers" 에러
```

```
원인: MaxPeers 설정값 초과

해결방법:

- MaxPeers 값 증가

- 불필요한 연결 정리

- 연결 품질 개선
```

3. 메시지 압축 문제

```
증상: 메시지 디코딩 실패

원인: Snappy 압축/해제 오류

해결방법:

- 프로토콜 버전 확인

- 압축 설정 동기화

- 메시지 크기 제한 확인
```

디버깅 도구

```
// 연결 상태 모니터링

func (srv *Server) PeerCount() int {

var  count  int

srv.peerOp <- func(peers  map[enode.ID]*Peer) {

count = len(peers)

}

<-srv.peerOpDone
```

```
return count
}

// 프로토콜 통계
func (p *Peer) Caps() []Cap {
return p.rw.caps
}

func (p *Peer) RemoteAddr() net.Addr {
return p.rw.fd.RemoteAddr()
}
```

성능 튜닝

연결 관리 최적화:

```
config := p2p.Config{
MaxPeers: 100, // 환경에 맞게 조정
MaxPendingPeers: 20, // 대기 연결 수
DialRatio: 3, // 아웃바운드 비율
ListenAddr: ":30303", // 리스닝 주소
}
```

메모리 사용량 최적화:

- 메시지 버퍼 크기 조정
- 연결 풀 크기 제한
- 가비지 컬렉션 주기 최적화

이 분석을 통해 geth의 devp2p 프로토콜 스택이 어떻게 안전하고 효율적인 P2P 통신을 제공하는지, 그리고 RLPx 암호화와 프로토콜 멀티플렉싱이 어떻게 구현되어 있는지 완전히 이해할 수 있습니다.